Multi-Objective Constraint Inference using Inverse reinforcement learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Constraint inference is widely considered essential to align reinforcement learning agents with safety boundaries and operational guidelines by observing expert demonstrations. However, existing approaches typically assume homogeneous demonstrations (i.e., generated by a single expert or multiple experts with identical objectives). They also have limited ability to capture individual preferences and often suffer from computational inefficiencies. In this paper, we introduce Multi-Objective Constraint Inference (MOCI), a novel framework designed to jointly extract shared constraints and individual preferences from heterogeneous expert trajectories, where multiple experts pursue different objectives. MOCI effectively models and learns from diverse, and potentially conflicting, behaviors. Empirical evaluations demonstrate that MOCI significantly outperforms existing baselines, achieving improved predictive performance, and maintaining competitive computational efficiency on a standard grid-world benchmark. These results establish MOCI as an accurate, flexible, and computationally practical approach for real-world constraint inference and preference learning tasks.

1 Introduction

Inverse Reinforcement Learning (IRL) is a framework for learning the underlying objectives of an expert by observing their behavior [1]. Instead of manually specifying a reward function, IRL infers the reward that best explains the expert's demonstrated actions. Traditionally, the intent of an agent is modeled by a learned reward function, however, accurately matching the expert's behavior can make the reward overly complex and brittle to small changes. Instead of capturing complex and diverse motivations of the expert in a single reward function, expert behavior can be more easily explained by jointly learning a simple reward function and inferring a set of clear hard constraints [2].

Although jointly learning constraints and reward functions yield simpler reward models that better explain expert trajectories, existing work [3, 4] suffers from two restrictive assumptions: (1) the dataset is homogeneous, i.e. generated by a single type of expert and (2) the expert's reward function is completely known *a priori*. These assumptions are problematic in real-world settings where observed data typically emerge from a heterogeneous population of experts, and the reward function is not known. For instance, in urban driving, different drivers exhibit varying driving styles, ranging from aggressive to cautious, yet all are subject to the same shared physical and legal constraints, such as speed limits and lane boundaries. At the same time, drivers have different driving and navigation preferences such as reaching the destination with minimum distance, or taking a longer route with more pleasant sceneries.

In this paper, we address the challenge of inferring shared constraints and learning individual preferences from heterogeneous expert demonstrations. We propose Multi-Objective Constraint Inference (MOCI), a novel framework to jointly recover shared constraints and preferences from unlabeled demonstrations generated by multiple experts with varying preferences and in an environment with

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

hard constraints shared across all experts.
MOCI iteratively clusters expert trajectories while learning personalized reward weights. Based on these clusters (groups), MOCI then identifies shared constraints C by evaluating each state using the joint log-likelihood method [5]. A hard constraint is identified for a state only if its removal from the feasible environment increases the likelihood of the observed demonstrations across all clusters simultaneously. Hence, MOCI jointly learns shared constraints and personalized reward weights by applying maximum entropy inverse reinforcement learning to each group, allowing the separation of individual expert preferences from shared environmental constraints.
We evaluate our approach on a multi-objective GridWorld environment, a well-established benchmark for sequential decision-making problems. This controlled setting allows us to systematically validate the proposed method.
The remainder of this paper is organized as follows: Section 2 formalizes the preliminaries and foundational concepts upon which the framework is built, including Constrained Markov Decision Processes (CMDPs) and Maximum Entropy Inverse Reinforcement Learning (IRL). Section 3 introduces the proposed Multi-Objective Constraint Inference (MOCI) algorithm and provides a detailed analysis of its computational complexity. Section 4 details the experimental setup using a simulated heterogeneous Gridworld environment and discusses the empirical results of the framework. Section 5 provides a performance comparison between MOCI and existing baseline techniques. Finally, Section 6 summarizes the study's conclusions and discusses current limitations.

2 Preliminaries

In this section, we formalize the foundational concepts upon which the Multi-Objective Constraint Inference (MOCI) framework is built. We define the environments, the structure of the agent's objectives, and the methodologies used to infer those objectives from the data.

2.1 Constrained Markov Decision Processes

A standard Markov Decision Process (MDP) is defined as a tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \gamma, R \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{T}(s' \mid s, a) \in \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ is the transition probability distribution, $\gamma \in [0, 1)$ is the discount factor, $R(s, a) \in \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function [6]. The behavior of an agent is defined by a policy $\pi(a \mid s) \in \mathcal{S} \to \Delta(\mathcal{A})$, which assigns states to a probability distribution over actions.
A *Constrained Markov Decision Process* (CMDP) [7] extends this framework by restricting the set of allowable policies. Although CMDP can be formulated using cost functions and budget thresholds, we focus on the context of hard environmental constraints (such as walls or strict physical limitations), defined as a set of forbidden or unsafe states denoted $C \subset \mathcal{S}$ following recent related work by Kim et al. [8], Qadri et al. [9].
The CMDP is thus augmented to $\mathcal{M}_C = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \gamma, R, C \rangle$.
A trajectory $\xi = \{(s_0, a_0), (s_1, a_1), \ldots, (s_T, a_T)\}$ of length $H \in \mathbb{N}$ is considered valid if and only if it does not violate any constraints in C [10]. This is formalized using a binary indicator function:

$$\mathbb{I}^C(\xi) = \begin{cases} 1 & \text{if } s_t \notin C \text{ for all } t \in \{0, \ldots, H\} \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

2.2 Multi-Objective MDP

A *Multi-Objective Markov Decision Process* (MOMDP) extends traditional MDPs with a vectorial reward function $\boldsymbol{R} \in \mathcal{S} \times \mathcal{A} \to \mathbb{R}^d$ to model environments with d potentially competing objectives (e.g., minimizing travel time while maximizing safety) [11, 12]. This vectorial reward can be collapsed into a scalar reward if the particular way in which these multiple objectives are to be combined for some particular individual or use case, are known. In case this so-called *scalarization* function is linear in the objectives, we refer to its weights $w := [w_1, \ldots, w_p]$ as the preferences of that individual such that their reward function $R = w^\top \boldsymbol{R}$.

In practice, this is often expressed via a feature formulation where $\phi(s, a) \in \mathbb{R}^d$ represents a vector of state-action features, and an agent's specific preference is defined by a weight vector w. The scalarized reward [13] for an agent with preference w can be defined as:

$$R_w(s, a) = w^\top \phi(s, a) \tag{2}$$

In a heterogeneous multi-agent setting [14], different experts $k \in \{1, \ldots, K\}$ share the same state-action features ϕ but possess distinct, private preference weights w_k, leading to diverse optimal policies within the same underlying environment.

2.3 Inverse Reinforcement Learning

Inverse Reinforcement Learning (IRL) addresses the problem of extracting an agent's underlying reward function given its demonstrated behavior [15]. Formally, we assume access to the MDPs transition function but not its reward function, denoted as $\mathcal{M} \setminus R$, and a dataset of expert trajectories $\mathcal{D} = \{\xi_1, \ldots, \xi_N\}$ generated by an expert policy π_E.

The goal of IRL is to find a reward function R^* such that the expert's policy π_E is optimal[16]. If the reward is parameterized linearly as $R(s, a) = w^\top \phi(s, a)$, the IRL problem can be framed as finding a weight vector w^* such that the expected feature counts of the expert match the expected feature counts of a policy optimizing w^*:

$$\mathbb{E}_{\pi_E}\left[\sum_{t=0}^{T} \gamma^t \phi(s_t, a_t)\right] = \mathbb{E}_{\pi_{w^*}}\left[\sum_{t=0}^{T} \gamma^t \phi(s_t, a_t)\right] \tag{3}$$

However, this problem is inherently ill-posed, as multiple reward functions (including a trivial reward of all zeros) can explain the same behavior.

2.4 Maximum Entropy IRL

To resolve the ambiguity of the IRL problem, [17] introduced *Maximum Entropy Inverse Reinforcement Learning* (MaxEnt IRL). MaxEnt IRL applies the principle of maximum entropy to select the probability distribution over trajectories that matches the expert's empirical feature expectations while making no other assumptions (i.e., being as random as possible otherwise).

Under the MaxEnt framework, the probability of an agent choosing a specific trajectory ξ is exponentially proportional to the total accumulated reward of that trajectory:

$$P(\xi \mid w) = \frac{1}{Z(w)} \exp\left(\sum_{(s,a)\in\xi} w^\top \phi(s, a)\right) = \frac{1}{Z(w)} e^{R_w(\xi)} \tag{4}$$

where $Z(w)$ is the partition function, representing the integral (or sum) over all possible trajectories originating from the start state:

$$Z(w) = \sum_{\xi' \in \Xi} e^{R_w(\xi')} \tag{5}$$

The reward weights w are then found by maximizing the log-likelihood of the demonstrated trajectories $\mathcal{D}$:

$$\mathcal{L}(w) = \sum_{i=1}^{|\mathcal{D}|} \log P(\xi_i \mid w) = \sum_{i=1}^{|\mathcal{D}|} \left(w^\top \phi(\xi_i) - \log Z(w)\right) \tag{6}$$

The gradient of this log-likelihood neatly reduces to the difference between the empirical feature counts of the demonstrations and the expected feature counts under the current weight vector w, allowing for efficient optimization via gradient ascent.

3 Multi-Objective Constraint Inference (MOCI)

In this section, we present our approach for jointly learning shared constraints and individual preferences. We also provide a theoretical analysis of the computational complexity of the proposed algorithm, highlighting its scalability with respect to the number of states, actions, and demonstrations.

3.1 Approach

Let $\mathcal{D} = \{\xi_1, \ldots, \xi_N\}$ be a dataset of demonstrated trajectories of maximum length H. We assume a *Constraint Multi-Objective Markov Decision Process* (CMOMDP) $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \gamma, \boldsymbol{R}, C \rangle$ and the

existence of K latent expert types (clusters), where each type $k \in \{1, \ldots, K\}$ is characterized by a specific preference weight vector w_k such that $R_k = w_k^\top \boldsymbol{R}$, and a prior probability $\pi_k = P(k)$. Crucially, the demonstrations are not labelled by their associated expert type $k \in K$ as inferring expert preferences is a key goal of this work.

Although experts differ in their preferences, all agents share the same state-action feature function ϕ, and operate subject to a shared set of hard physical constraints, denoted by C. Following the Maximum Entropy Inverse Reinforcement Learning (MaxEnt IRL) framework by Ziebart et al. [17], the probability of observing a specific trajectory ξ, given that it was generated by an expert of type k subject to constraints C, is defined as:

$$P(\xi \mid C, w_k) = \frac{1}{Z(C, w_k)} e^{R_{w_k}(\xi)} \mathbb{I}^C(\xi) \tag{7}$$

where, $R_{w_k}(\xi) = \sum_{(s,a)\in\xi} w_k^\top \phi(s, a)$ is the cumulative reward of the trajectory under preference w_k. $Z(C, w_k)$ is the partition function over all feasible paths in the constrained Markov Decision Process (MDP). The detailed analysis and proof are given in the appendix-B. $\mathbb{I}^C(\xi)$ is an indicator function that equals 1 if the trajectory ξ does not violate any constraint in C and 0 otherwise as presented in equation (1). Marginalizing over the latent assignment of demonstrations to expert types, the likelihood of a single demonstration is:

$$P(\xi \mid C, \{w_k\}, \{\pi_k\}) = \sum_{k=1}^{K} \pi_k \frac{e^{R_{w_k}(\xi)}}{Z(C, w_k)} \mathbb{I}^C(\xi) \tag{8}$$

Our objective is to jointly infer the shared constraints C, the latent reward weights $\{w_k\}$, and the priors $\{\pi_k\}$ by maximizing the joint log-likelihood of the dataset $\mathcal{D}$:

$$\mathcal{L}(C, \{w_k\}, \{\pi_k\}) = \sum_{i=1}^{|\mathcal{D}|} \log \left(\sum_{k=1}^{K} \pi_k \frac{e^{R_{w_k}(\xi_i)}}{Z(C, w_k)} \mathbb{I}^C(\xi_i) \right) \tag{9}$$

Because the assignments of the demonstrations to agent types or clusters are unobserved, directly optimizing this joint log-likelihood is intractable. We therefore optimize equation (9) using an Expectation-Maximization (EM) approach [18], which alternates between estimating the posterior probability of cluster assignments and updating the model parameters alongside the constraint set.

In the Expectation step (E-step), we fix the current constraints C, weights $\{w_k\}$, and priors $\{\pi_k\}$, and compute the responsibility $\gamma_{i,k}$, which represents the posterior probability that trajectory ξ_i was generated by expert k:

$$\gamma_{i,k} = \frac{\pi_k P(\xi_i \mid C, w_k)}{\sum_{j=1}^{K} \pi_j P(\xi_i \mid C, w_j)} \tag{10}$$

In the Maximization step (M-step), we update the parameters to maximize the expected log-likelihood over the entire dataset. The cluster priors are updated as the empirical mean of the responsibilities, such that

$$\pi_k = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \gamma_{i,k} \tag{11}$$

To update the reward weights w_k for each cluster, we perform gradient ascent where the gradient for trajectory i and cluster k is weighted by the responsibility $\gamma_{i,k}$, ensuring that the weights adapt to the trajectories probabilistically assigned to that cluster:

$$\nabla_{w_k} \mathcal{L} = \sum_{i=1}^{|\mathcal{D}|} \gamma_{i,k} \left(\phi(\xi_i) - \mathbb{E}_{P(\xi|C,w_k)}[\phi(\xi)] \right) \tag{12}$$

where $\mathbb{E}_{P(\xi|C,w_k)}[\phi(\xi)]$ is the expected feature count under the current constraints and cluster weights. $w_k \leftarrow w_k + \alpha \nabla_{w_k} \mathcal{L}$

Finally, to update the shared constraints C, we execute a greedy search over the set of candidate constraints (states not visited by any expert in $\mathcal{D}$). For a candidate constraint c, we define its score as the joint log-likelihood evaluated with the augmented constraint set;

$$\text{Score}(c) = \mathcal{L}(C \cup \{c\}, \{w_k\}, \{\pi_k\}) \tag{13}$$

The algorithm iteratively adds the candidate $\hat{c} = \arg\max_c \text{Score}(c)$ to C.
This greedy addition terminates when the reduction in Kullback-Leibler (KL) divergence falls below a specified threshold d_{DKL}, which is mathematically equivalent to stopping when the increase in log-likelihood satisfies;

$$\mathcal{L}(C \cup \{\hat{c}\}, \cdot) - \mathcal{L}(C, \cdot) \leq d_{DKL} \tag{14}$$

This thresholding mechanism serves as a regularizer, preventing the algorithm from overfitting to noise by rejecting constraints that yield only marginal improvements to the model's likelihood.

The algorithm for MOCI is presented in appendix -D. The goal of the algorithm is to employs an Expectation-Maximization (EM) approach to infer shared constraints and multi-objective preferences.

3.2 Computational Complexity

The MOCI algorithm calculates the likelihood of each trajectory in $\mathcal{D}$ demonstrations belonging to each group $k \in K$. It computes partition function Z, which iterates through the maximum horizon (trajectory length) H, checking all actions $|\mathcal{A}|$ for all valid states $|\mathcal{S}|$. Which yields complexity:

$$\mathcal{O}_E(K \cdot H \cdot (|\mathcal{S}| \cdot |\mathcal{A}| + |\mathcal{D}|)) \tag{15}$$

For each cluster K, the algorithm performs I_{IRL} gradient descent steps. In each step, it recomputes Z to find the expected feature counts and compares them to the empirical features of the demonstrations. It can be shown as:

$$\mathcal{O}_M(K \cdot I_{IRL} \cdot H \cdot (|\mathcal{S}| \cdot |\mathcal{A}| + |\mathcal{D}|)) \tag{16}$$

The algorithm also tests unvisited candidate states to see if adding them to the constraint set $\mathcal{C}$ reduces the log-likelihood by less than threshold (d_{DKL}). Let $\mathcal{C}_{added}$ be the number of constraints successfully inferred in a single step. For each added constraint, the algorithm tests a subset of candidates (M_{test}).

$$\mathcal{O}_\mathcal{C}(\mathcal{C}_{added} \cdot M_{test} \cdot K \cdot |\mathcal{S}| \cdot |\mathcal{A}| \cdot H) \tag{17}$$

To find the total computational cost, we sum the complexities of all the steps (Expectation step $\mathcal{O}_E$, Weights $\mathcal{O}_M$, and Constraints $\mathcal{O}_\mathcal{C}$), and multiply by the total number of iterations (I_{Iter}).

$$\text{Cost} = \mathcal{O}\Big(I_{Iter} \cdot K \cdot |\mathcal{S}| \cdot |\mathcal{A}| \cdot H \cdot (1 + I_{IRL} + \mathcal{C}_{added} \cdot M_{test})\Big) \tag{18}$$

To simplify complexity equation, we isolate the terms that grow the fastest as the problem scales toward infinity. In Inverse Reinforcement Learning, the hyper-parameters ($I_{Iter}, K, I_{IRL}, M_{test}, \mathcal{C}_{added}$) can be treated as constants. Furthermore, the size of the state-action space multiplied by the horizon ($|\mathcal{S}| \cdot |\mathcal{A}| \cdot H$) will always asymptotically dominate the number of sampled demonstrations ($|\mathcal{D}|$).
By dropping the constants and non-dominant terms, the total theoretical computational cost reduces to:

$$\text{Cost} = \mathcal{O}(|\mathcal{S}| \cdot |\mathcal{A}| \cdot H) \tag{19}$$

The cost of MOCI is highly dependent on the total number of states $|\mathcal{S}|$, the total number of actions per state $|\mathcal{A}|$, and the horizon (maximum length of a trajectory) H. For a specific environment, e.g., Gridworld, the total number of states $|\mathcal{S}| = N^2$, hence the MOCI algorithm scales quadratically with the grid dimensions.

4 Experiments: Heterogeneous Gridworld

To validate the Multi-Objective Constraint Inference (MOCI) framework, we designed a simulated multi-feature Gridworld environment. This environment serves as a controlled testbed for evaluating the MOCI algorithm.

4.1 Environment Setup

The environment is modeled as a deterministic Markov Decision Process (MDP) on a $N \times N$ grid as shown in figure-1. The state space $\mathcal{S}$ consists of grid cells, and the action space $\mathcal{A}$ allows for standard movement: {*Up, Down, Left, Right, Stay*}. The environment features a "Start" state in the top-left corner and a "Goal" state in the bottom-right corner.

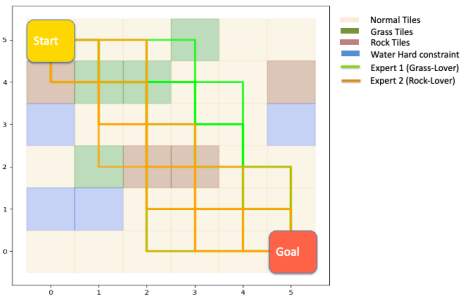

Figure 1: Demonstration of the MOCI algorithm in a 6x6 Gridworld. Ground-truth environment with water hard constraints (blue) and expert trajectories for a Grass-Lover (lime) and Rock-Lover (orange).

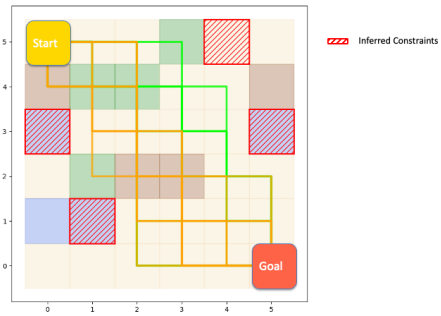

Figure 2: MOCI-inferred constraints (red hatched), showing successful recovery of true constraints alongside false positives in unvisited states.

Each state $s \in \mathcal{S}$ is assigned a categorical feature vector $\phi(s) \in \{0, 1\}^4$ that represents four different types of terrain: grasslands (green), Rocks(brown), Water (blue) and Normal (light beige) tiles. Exactly one feature is active for any given state. The true underlying environment contains a set of impassable obstacles (blue water tiles in this case) that represent our set of ground-truth hard constraints C^*. Transitions into states $s \in C^*$ are strictly prohibited in the environment dynamics. To simulate a heterogeneous population, we defined two distinct expert types with divergent reward preferences, parameterized by their ground-truth weight vectors w_k^*:

1. **Expert 1 (Grass-Lover):** Highly prefers traveling on the Grass tiles
2. **Expert 2 (Rock-Lover):** Highly prefers traveling on the Rock tiles

The agent's objectives are to reach the goal state by taking the shortest path while navigating their preferred terrain. Both experts share the same terminal goal state and are fundamentally restricted by the Water tiles (C^*).
To generate the demonstration dataset $\mathcal{D}$, we computed the optimal Maximum Entropy policy π_k^* for each expert using Soft Value Iteration based on their respective reward weights w_k^*. We then sampled $|\mathcal{D}_1| = 10$ trajectories from Expert 1 and $|\mathcal{D}_2| = 10$ trajectories from Expert 2. The final dataset provided to the inference algorithms was an unlabeled, randomly shuffled mixture of these 20 trajectories, $\mathcal{D} = \mathcal{D}_1 \cup \mathcal{D}_2$. In Figure-1, the trajectory of Expert 1 (Grass-Lover) is shown in lime, while the trajectory of Expert 2 (Rock-Lover) is shown in orange.

4.2 Results

All results reported in this paper were obtained by running experiments on a *MacBook M3* using *Python 3.11.5*. The figure-2 visualizes the constraints inferred by the MOCI algorithm after observing the expert trajectories shown in figure-1. The algorithm successfully places "Inferred Constraints" (marked by red hatched boxes) over the blue water tiles. By observing that neither the grass-lover nor the rock-lover ever stepped on these tiles, MOCI correctly deduced they are forbidden.
The red hatched boxes also appear on several non-water tiles (e.g., a normal tile on the top right). This occurs because these specific location was never visited by either expert. The algorithm conservatively (and incorrectly) inferred that these unvisited states must also be hard constraints. It shows that while MOCI can successfully learn heterogeneous preferences and recover ground-truth constraints (the water), limited dataset coverage can lead the algorithm to hallucinate constraints in regions of the state space that were simply ignored by the expert's specific goals.
The figure-3 illustrates the MOCI algorithm's capability to untangle and learn the distinct underlying reward functions of different types of experts from a shared environment. The visualization is split into two side-by-side bar charts, each representing a specific expert profile evaluated across four terrain features: Sand, Grass, Rocks, and Water. This left-hand chart compares the original Ground Truth weights (light gray bars) against the MOCI Learned weights (green bars). The algorithm

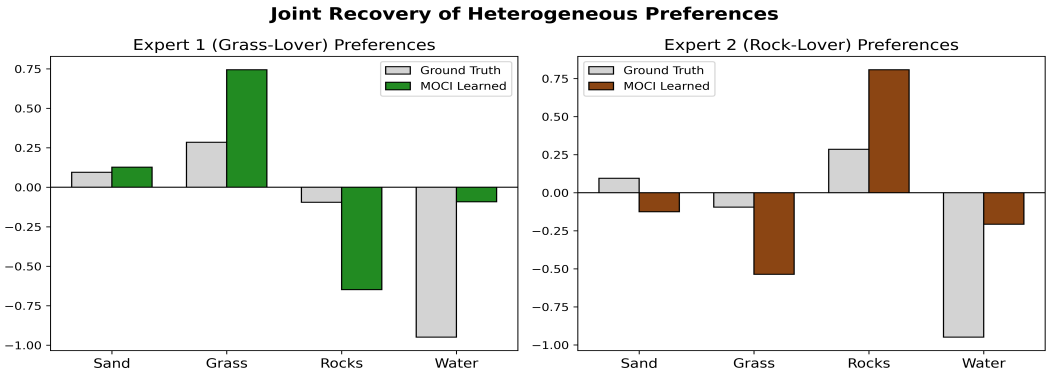

Figure 3: Joint recovery of heterogeneous preferences using the MOCI algorithm. The chart compares the ground-truth reward weights with the learned weights across four terrain features for two distinct expert profiles (a Grass-Lover and a Rock-Lover).

accurately captures the expert's defining trait: a strong positive preference for Grass. It also correctly identifies the negative polarity for Rocks and Water, meaning the algorithm learned that the expert avoids these terrains. The right-hand chart contrasts the Ground Truth weights (light gray bars) with the MOCI Learned weights (brown bars) for the second expert. Here, the algorithm successfully identifies the opposite behavior, showing a strong positive preference for Rocks. It correctly assigns negative weights to Grass and Water, capturing the expert's specific aversions.

Tuning Threshold d_{DKL}: The adjustment of the threshold value is arguably the most critical step in the implementation of MOCI. The threshold (d_{DKL} dictates how much the likelihood of a trajectory must drop before the algorithm considers a state to be a hard constraint. If it is too high, the algorithm misses the real walls (false negatives). If it is too low, the algorithm looks at a perfectly navigable tile that the experts simply disliked and declares it a physical wall (a false positive). We have presented some of the factors on which the performance of MOCI is really dependent in Appendix-C.

To ensure that the threshold remains invariant with the size of the dataset $|\mathcal{D}|$, we normalize the joint log-likelihood to represent the average information gained per trajectory:

$$L_{avg} = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \log P(\xi_i | w_k, C) \tag{20}$$

By evaluating the normalized change, ΔL_{avg}, the divergence threshold d_{DKL} can be robustly set to a constant value across varying expert population sizes. Established literature [3] suggests an effective range of $0.01 \leq d_{DKL} \leq 2$ for this parameter. Figure-4 illustrates the False Positive Rate (FPR) with respect to the four different thresholds ($d_{DKL} \in \{0.01, 0.05, 0.5, 2.0\}$). The performance of the algorithm is evaluated on different number of Expert Demonstrations ($|\mathcal{D}|$). Despite the different threshold values, the FPR does not escalate as the number of expert demonstrations increases. Instead, the metric remains highly stable and shows a trend toward convergence. When the algorithm receives

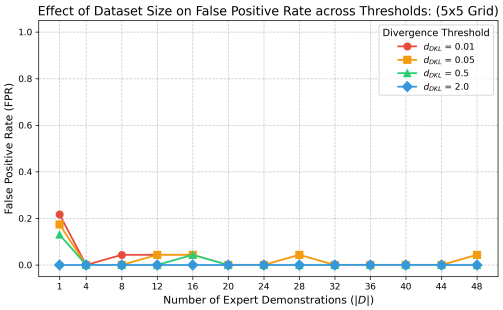

Figure 4: Effect of the number of expert demonstrations ($|\mathcal{D}|$) on the False Positive Rate (FPR) during constraint inference evaluated across four thresholds (d_{DKL}) in a 5x5 Gridworld.

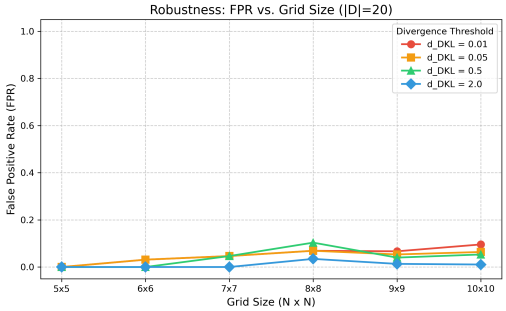

Figure 5: Robustness of the algorithm's False Positive Rate (FPR) relative to environmental scaling (grid sizes from 5x5 to 10x10) given a fixed dataset of $|D| = 20$ expert demonstrations.

very few demonstrations (e.g. $|\mathcal{D}| = 1$), there is a higher degree of uncertainty. This lack of data causes the algorithm to over-infer constraints, resulting in a higher FPR. As the number of expert demonstrations increases, the FPR drops and converges. The highest threshold ($d_{DKL} = 2.0$, represented by the blue line) consistently yields the lowest False Positive Rate for this particular gird size. This proves that the algorithm successfully normalizes the log-likelihood against the dataset size, preventing metric inflation as more data are introduced.

Robustness: Figure 5 presents an evaluation of the algorithm's robustness in terms of its False Positive Rate (FPR) as the environment scales (grid sizes), while keeping the amount of expert data fixed at $|\mathcal{D}| = 20$ demonstrations.

As the grid scales from 5×5 to 10×10, the total number of states increases quadratically (from 25 to 100). Consequently, a fixed dataset of 20 trajectories covers a progressively smaller percentage of the environment. Because the expert's coverage becomes sparser in larger grids, the algorithm's uncertainty increases slightly, leading to a minor upward trend in the FPR. Additionally, despite the state space quadrupling in size, the FPR remains remarkably low.

Scalability effect on runtime: Figure-6 illustrates the execution time (in seconds) of the MOCI algorithm as the complexity of the environment increases from a 5×5 grid to a 10×10 grid. The blue line (short trajectory) represents an optimal direct path to the goal, where the horizon scales roughly twice the dimension of the grid ($H \approx 2N$). The red line (long trajectory) represents a sub-optimal or highly exploratory path, where the horizon could scale at five times the grid dimension ($H \approx 5N$). For optimal paths ($2N$), the computational cost increases at a manageable rate.

In a 5×5 grid, the execution takes roughly 0.3 seconds, and it smoothly increases to just under 1.6 seconds for a 10×10 grid. For exploratory paths ($5N$), the algorithm requires significantly more computation. The execution time curves increase much more sharply, starting around 0.4 seconds for a 5×5 grid but rising to nearly 4.7 seconds for a 10×10 grid. This graph perfectly validates the theoretical computational cost previously defined as $\mathcal{O}(N^2 \cdot |\mathcal{A}| \cdot H)$.

In summary, these findings validate MOCI as a uniquely robust, scalable, and computationally viable framework for extracting shared constraints from complex, heterogeneous expert behaviors.

5 Comparison with existing techniques

To rigorously evaluate the efficacy of the proposed Multi-Objective Constraint Inference (MOCI) framework, it is imperative to benchmark its performance against established state-of-the-art techniques. Table-1 presents the performance comparison with existing techniques. It compares the MOCI techniques with the two existing baselines, Maximum Likelihood Constraint Inference (MLCI) [3] and Inverse Constrained Reinforcement Learning (ICRL) [4] across four evaluation metrics.

Constraint mean squared error (CMSE) is computed as the mean squared error between the true constraint function and the recovered constraint function.

Run-time represents the execution or computational time required by the technique, including training and inferring the constraints.

Trajectory Types indicates whether the model handles "Homogeneous" or "Heterogeneous" data.

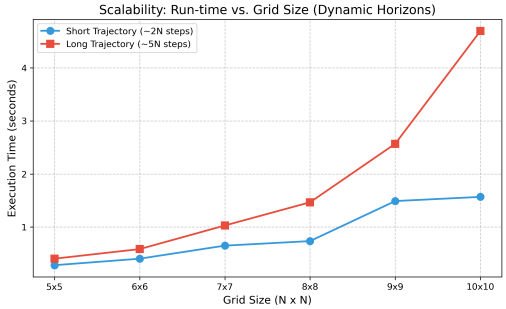

Figure 6: Scalability analysis of run-time versus grid size with dynamically scaling trajectory lengths. The execution time exhibits a steepening curve, effectively validating the theoretical complexity $\mathcal{O}(|\mathcal{S}| \cdot |\mathcal{A}| \cdot H)$.

Table 1: Performance Comparison with existing techniques

Techniques	CMSE	Run-time	Trajectory Types	Preference Learning
MLCI [3]	0.25	0.2927	Homogeneous	No
ICRL [4]	0.36	8.90	Homogeneous	No
MOCI (our)	0.027	0.6917	Heterogeneous	Yes

Preference Learning is a boolean metric indicating whether the technique utilizes preference learning ("Yes" or "No").

We measured these performace matrices over a 5×5 gridworld and results are summarized in Table-1. Our proposed MOCI framework demonstrates superior predictive accuracy, achieving a Mean Squared Error (MSE) of 0.027, which represents a substantial improvement over the baseline techniques MLCI (0.25) and ICRL (0.36). Although MLCI remains the most computationally efficient model with a run-time of 0.2927, MOCI maintains a highly competitive run-time of 0.6917. This is because in MLCI, there are no explicit weight updates during the constraint inference loop. The nominal MDP has the expert's reward function completely known *a priori*. The only thing that changes is the set of forbidden states. On the other hand, MOCI maintains a list of weights and it performs gradient descent to actively update the reward parameters for each cluster based on the responsibilities calculated in the each step. ICRL has a comparatively longer run-time (8.90), mainly due to its neural network–based soft parameterization of the indicator set over constrained trajectories.

Furthermore, unlike MLCI and ICRL, which are restricted to homogeneous trajectory types and lack preference learning capabilities, MOCI is uniquely able to process heterogeneous trajectories while explicitly incorporating preference learning. These advantages highlight MOCI's enhanced flexibility and robustness without sacrificing computational efficiency.

6 Conclusion

In this study, we presented MOCI, a novel Multi-Objective Constraint Inference framework that successfully addresses the limitations of current constraint learning techniques. By leveraging preference learning, MOCI is uniquely process heterogeneous trajectory data, allowing it to accurately capture diverse expert behaviors and nuanced multi-objective trade-offs. Our comparative analysis reveals that MOCI achieves superior predictive accuracy, reducing the Mean Squared Error to 0.027. This is a substantial improvement over established baselines. Crucially, MOCI achieves this high accuracy while bypassing the severe computational overhead associated with iterative reinforcement learning loops, resulting in highly competitive execution times. These findings highlight MOCI's potential as a scalable and robust tool for safe reinforcement learning in complex environments where expert data is varied and multifaceted.

Limitations:

1. MOCI currently requires the hyperparameter K (the number of distinct expert types) to be defined a priori. In real-world, unannotated datasets, the exact number of heterogeneous behaviors is rarely known in advance. If K is set incorrectly, MOCI may either force distinct behaviors into a single distorted reward function or split a single behavior into redundant clusters.
2. The MOCI focuses strictly on hard constraints (states that are absolutely forbidden). It does not natively support the inference of soft constraints (e.g., a speed limit that an expert might occasionally violate but generally avoids) or probabilistic chance constraints.

Future work:

1. To eliminate the need for a predefined number of expert types, the EM clustering step could be replaced with Bayesian non-parametric methods, such as a Dirichlet Process Mixture Model (DPMM) [19]. This would allow the algorithm to automatically deduce the optimal number of distinct expert profiles (K) directly from the complexity and diversity of the demonstration data.
2. This work can be extended to real-world healthcare applications. In cancer treatment, patients are typically subject to shared clinical constraints, such as treatment guidelines, toxicity limits, and eligibility criteria for specific therapies (e.g., chemotherapy, radiotherapy). However, individual preferences can vary substantially. Some patients may prioritize prolonging survival, even at the cost of aggressive treatments and severe side effects, while others may prefer to preserve quality of life and opt for less intensive treatments. Furthermore, treatment effects are highly heterogeneous [20], as patients may respond differently to the same therapy due to clinical, biological, and personal factors. By leveraging MOCI to learn shared constraints, we aim to optimize treatment strategies that align with individual patient preferences while adhering common constraints and safety boundaries.

References

[1] Syed Ihtesham Hussain Shah, Antonio Coronato, Muddasar Naeem, and Giuseppe De Pietro. Learning and assessing optimal dynamic treatment regimes through cooperative imitation learning. *IEEE Access*, 10:78148–78158, 2022.

[2] Dexter RR Scobee and S Shankar Sastry. Maximum likelihood constraint inference for inverse reinforcement learning. *arXiv preprint arXiv:1909.05477*, 2019.

[3] David L McPherson, Kaylene C Stocking, and S Shankar Sastry. Maximum likelihood constraint inference from stochastic demonstrations. In *2021 IEEE conference on control technology and applications (CCTA)*, pages 1208–1213. IEEE, 2021.

[4] Shehryar Malik, Usman Anwar, Alireza Aghasi, and Ali Ahmed. Inverse constrained reinforcement learning. In *International conference on machine learning*, pages 7390–7399. PMLR, 2021.

[5] Siliang Zeng, Chenliang Li, Alfredo Garcia, and Mingyi Hong. Maximum-likelihood inverse reinforcement learning with finite-time guarantees. *Advances in Neural Information Processing Systems*, 35:10122–10135, 2022.

[6] Rahul Singh, Abhishek Gupta, and Ness B Shroff. Learning in constrained markov decision processes. *IEEE Transactions on Control of Network Systems*, 10(1):441–453, 2022.

[7] Akifumi Wachi and Yanan Sui. Safe reinforcement learning in constrained markov decision processes. In *International Conference on Machine Learning*, pages 9797–9806. PMLR, 2020.

[8] Konwoo Kim, Gokul Swamy, Zuxin Liu, Ding Zhao, Sanjiban Choudhury, and Steven Z Wu. Learning shared safety constraints from multi-task demonstrations. *Advances in Neural Information Processing Systems*, 36:5808–5826, 2023.

[9] Mohamad Qadri, Gokul Swamy, Jonathan Francis, Michael Kaess, and Andrea Bajcsy. Your learned constraint is secretly a backward reachable tube. *Reinforcement Learning Journal*, 6: 478–492, 2025.

[10] Andreas Schlaginhaufen and Maryam Kamgarpour. Identifiability and generalizability in constrained inverse reinforcement learning. In *International conference on machine learning*, pages 30224–30251. PMLR, 2023.

[11] Leon Barrett and Srini Narayanan. Learning all optimal policies with multiple criteria. In *Proceedings of the 25th international conference on Machine learning*, pages 41–47, 2008.

[12] Conor F Hayes, Roxana Rădulescu, Eugenio Bargiacchi, Johan Källström, Matthew Macfarlane, Mathieu Reymond, Timothy Verstraeten, Luisa M Zintgraf, Richard Dazeley, Fredrik Heintz, et al. A practical guide to multi-objective reinforcement learning and planning: Cf hayes et al. *Autonomous Agents and Multi-Agent Systems*, 36(1):26, 2022.

[13] Syed Ihtesham Hussain Shah, Giuseppe De Pietro, Giovanni Paragliola, and Antonio Coronato. Projection based inverse reinforcement learning for the analysis of dynamic treatment regimes. *Applied Intelligence*, 53(11):14072–14084, 2023.

[14] Yifan Zhong, Jakub Grudzien Kuba, Xidong Feng, Siyi Hu, Jiaming Ji, and Yaodong Yang. Heterogeneous-agent reinforcement learning. *Journal of Machine Learning Research*, 25(32): 1–67, 2024.

[15] Saurabh Arora and Prashant Doshi. A survey of inverse reinforcement learning: Challenges, methods and progress. *Artificial Intelligence*, 297:103500, 2021.

[16] Stephen Adams, Tyler Cody, and Peter A Beling. A survey of inverse reinforcement learning. *Artificial Intelligence Review*, 55(6):4307–4346, 2022.

[17] Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, Anind K Dey, et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pages 1433–1438. Chicago, IL, USA, 2008.

[18] Mona Hamidi, Mina Sheikhalishahi, and Fabio Martinelli. Privacy preserving expectation maximization (em) clustering construction. In *International Symposium on Distributed Computing and Artificial Intelligence*, pages 255–263. Springer, 2018.

[19] Scott Niekum and Andrew Barto. Clustering via dirichlet process mixture models for portable skill discovery. *Advances in neural information processing systems*, 24, 2011.

[20] Haofeng Ye. Deep reinforcement learning-driven efficacy-toxicity balance optimization strategy for personalized drug combination in cancer patients. *Journal of Science, Innovation & Social Impact*, 1(1):307–317, 2025.

Appendices

A Notations and description

Table-2 summarizes the notation used in this paper along with their descriptions.

Table 2: Notations and Descriptions for Multi-Objective Constraint Inference (MOCI)

Notation	Description
$\mathcal{M}$	Markov Decision Process (MDP)
$\mathcal{S}$	State space
$\mathcal{A}$	Action space
$\mathcal{T}(s'\|s,a)$	Transition probability distribution
γ	Discount factor
$R(s,a)$	Reward function
$\pi(a\|s)$	Policy
C	Set of forbidden or unsafe states (constraints)
$\mathcal{M}_C$	Constrained Markov Decision Process (CMDP)
ξ	Trajectory of state-action pairs
H	Length of a trajectory
$\mathcal{I}_C(\xi)$	Binary indicator function for constraint violation
$\phi(s,a)$	Vector of state-action features
w	Preference weight vector for scalarization
$R_w(s,a)$	Scalarized reward function for preference w
$k \in \{1,...,K\}$	Latent expert types or clusters
$P(\xi\|w)$	Probability of a trajectory under Maximum Entropy IRL
$Z(w)$	Partition function (sum over all possible trajectories)
$\mathcal{D}$	Dataset of expert demonstrated trajectories
$\mathcal{L}(w)$	Log-likelihood of demonstrated trajectories
ΔL	Change in log-likelihood upon adding a candidate state to C
d_{DKL}	Divergence threshold parameter for constraint classification

B Constraint Muti-Objective Maximum Entropy IRL

Let ξ denote a trajectory and $f(\xi) \in \mathbb{R}^m$ its associated feature vector encoding m objectives. For each expert cluster k, preferences over objectives are represented by a weight vector $w_k \in \mathbb{R}^m$. The trajectory-level reward is defined as

$$R_{w_k}(\xi) = w_k^\top f(\xi). \tag{21}$$

We assume the presence of a set of hard constraints C such that any trajectory violating these constraints is considered infeasible. This is encoded using an indicator function

$$\mathbb{I}^C(\xi) = \begin{cases} 1, & \text{if } \xi \text{ satisfies all constraints in } C, \\ 0, & \text{otherwise.} \end{cases} \tag{22}$$

The goal of Maximum Entropy Inverse Reinforcement Learning is to find a probability distribution $P(\xi \mid C, w_k)$ over feasible trajectories that maximizes entropy while matching the empirical feature expectations of expert demonstrations. Formally, we solve

$$\max_{P(\xi)} \quad -\sum_{\xi} P(\xi) \log P(\xi) \tag{23}$$

$$\text{s.t.} \quad \sum_{\xi} P(\xi) = 1, \tag{24}$$

$$\sum_{\xi} P(\xi) f(\xi) = \hat{f}, \tag{25}$$

$$P(\xi) = 0 \quad \text{if } \mathbb{I}^C(\xi) = 0. \tag{26}$$

Introducing Lagrange multipliers λ and w_k for the normalization and feature-matching constraints, the Lagrangian becomes

$$\mathcal{L} = -\sum_{\xi} P(\xi) \log P(\xi) + \lambda \left(\sum_{\xi} P(\xi) - 1 \right) + w_k^\top \left(\sum_{\xi} P(\xi) f(\xi) - \hat{f} \right). \tag{27}$$

Taking the derivative of $\mathcal{L}$ with respect to $P(\xi)$ and setting it to zero yields

$$-\log P(\xi) - 1 + \lambda + w_k^\top f(\xi) = 0. \tag{28}$$

Solving for $P(\xi)$ gives

$$P(\xi) = \exp\big(w_k^\top f(\xi)\big) \exp(\lambda - 1). \tag{29}$$

The normalization constant, or partition function, is defined over feasible trajectories as

$$Z(C, w_k) = \sum_{\xi : \mathbb{I}^C(\xi) = 1} \exp\big(R_{w_k}(\xi)\big). \tag{30}$$

Substituting $\exp(\lambda - 1) = 1/Z(C, w_k)$ yields the final form:

$$P(\xi \mid C, w_k) = \frac{1}{Z(C, w_k)} \exp\big(R_{w_k}(\xi)\big), \tag{31}$$

for all ξ satisfying the constraints, and $P(\xi \mid C, w_k) = 0$ otherwise. The idea is taken from Maximum Entropy Inverse Reinforcement Learning by Zeibert et al.

C Sensitivity analysis of MOCI

The magnitude of the change in log-likelihood, ΔL, upon adding a candidate state to the constraint set C is intrinsically dependent on three environmental factors:

- **Dataset Size ($|D|$):** The total log-likelihood L is an unnormalized sum over all trajectories in the expert dataset. Consequently, the magnitude of ΔL scales linearly with the number of expert demonstrations.
- **State Space Complexity:** In environments with limited pathways (e.g., a heavily constrained Gridworld), removing a single valid state drastically reduces the partition function Z. Conversely, in highly redundant state spaces, the reduction in Z is minimal unless the state represents a critical navigational chokepoint.
- **State Centrality:** Unvisited states located on optimal paths yield a massive ΔL when constrained, whereas unvisited states in peripheral or suboptimal regions yield negligible changes to the partition function.

C.1 Ablation: latent mixture versus a single preference model

Heterogeneous demonstrations are modeled as a mixture over K latent linear preferences with shared hard constraints. A natural baseline is to *remove* mixture capacity by setting $K{=}1$, i.e., to explain the same pooled trajectories with a single MaxEnt component while retaining the remainder of the MOCI procedure (EM updates, constraint search, and the same hyperparameters otherwise). We instantiate MOCI with $K{\in}\{1, 2\}$ on the *same* unlabeled dataset $\mathcal{D}$ for each random demo seed: trajectories are drawn i.i.d. from two expert policies with distinct terrain preferences, then pooled and shuffled. This isolates whether mixture structure materially affects fit and constraint recovery when the generative process truly uses two preference types.

Table 3 summarizes means $\pm$ standard deviations over 50 independent demo seeds on the heterogeneous GridWorld protocol described above ($d_{\mathrm{DKL}}{=}0.05$, 10 EM iterations per run). We report average marginal log-likelihood per trajectory under the fitted model, constraint mean squared error (CMSE), precision and recall for forbidden states, their harmonic mean (F1), and false positive rate (FPR).

Table 3: Pooled heterogeneous demonstrations: single-preference MOCI ($K{=}1$) versus mixture MOCI ($K{=}2$). Means $\pm$ std. over 50 seeds.

Metric	$K{=}1$	$K{=}2$
Avg. marginal log-likelihood (per traj.)	-203.732 ± 0.146	-203.699 ± 0.156
CMSE (constraints)	0.0267 ± 0.0376	0.0233 ± 0.0324
Precision	0.857 ± 0.176	0.869 ± 0.157
Recall	0.935 ± 0.141	0.955 ± 0.109
F1 (forbidden states)	0.891 ± 0.155	0.907 ± 0.129
FPR	0.0219 ± 0.0270	0.0206 ± 0.0257

The ablation supports the hypothesis that mixture structure improves *explanation* of pooled heterogeneous data: $K{=}2$ achieves a systematic, statistically significant lift in marginal likelihood relative to $K{=}1$. Effects on overlap-based constraint scores are directionally favorable for $K{=}2$. Accordingly, we treat $K{=}1$ primarily as a likelihood-controlled sanity check on mixture capacity rather than as a substitute for the full model in heterogeneous regimes.

D MOCI Algorithm

The MOCI algorithm-1 employs an Expectation-Maximization (EM) style approach to infer shared constraints and multi-objective rewards jointly. During the E-step, it calculates the responsibility of each reward cluster for the observed expert trajectories. The M-step then updates the cluster priors, optimizes the reward weights for each objective via Maximum Entropy Inverse Reinforcement Learning, and greedily adds candidate constraints from unvisited states as long as they significantly improve the data likelihood without falling below the divergence threshold d_{DKL}.

Algorithm 1: Multi-Objective Constraint Inference (MOCI)

Input: MDP $\mathcal{M}$, Demonstrations $\mathcal{D} = \{\xi_1, \ldots, \xi_N\}$, Number of clusters K, threshold d_{DKL}, Priors $\{\pi_k\}_{k=1}^K$, Learning rate α

Output: Inferred constraints C, Reward weights $\{w_k\}_{k=1}^K$

Initialization:
$C \leftarrow \emptyset$
$\mathcal{C}_{cand} \leftarrow \{s \in \mathcal{S} \mid s \notin \xi \text{ for all } \xi \in \mathcal{D}\}$ // Unvisited states
Initialize w_k randomly, $\pi_k \leftarrow 1/K$ for $k \in \{1, \ldots, K\}$
repeat
  /* **E-Step: Evaluate Responsibilities** */
  for $k = 1$ **to** K **do**
    Compute partition function $Z(C, w_k)$ via backward pass
  end
  for $i = 1$ **to** N **do**
    for $k = 1$ **to** K **do**
      $\gamma_{i,k} \leftarrow \frac{\pi_k P(\xi_i|C,w_k)}{\sum_{j=1}^K \pi_j P(\xi_i|C,w_j)}$
    end
  end
  /* **M-Step A: Update Priors** */
  for $k = 1$ **to** K **do**
    $\pi_k \leftarrow \frac{1}{N} \sum_{i=1}^N \gamma_{i,k}$
  end
  /* **M-Step B: Update Reward Weights (MaxEnt IRL)** */
  for $k = 1$ **to** K **do**
    $\nabla_{w_k} \mathcal{L} \leftarrow \sum_{i=1}^N \gamma_{i,k} \left(\phi(\xi_i) - \mathbb{E}_{P(\xi|C,w_k)}[\phi(\xi)]\right)$
    $w_k \leftarrow w_k + \alpha \nabla_{w_k} \mathcal{L}$
  end
  /* **M-Step C: Update Shared Constraints** */
  while $\mathcal{C}_{cand} \neq \emptyset$ **do**
    $c^* \leftarrow \arg\max_{c \in \mathcal{C}_{cand}} \mathcal{L}(C \cup \{c\}, \{w_k\}, \{\pi_k\})$
    $\Delta\mathcal{L} \leftarrow \mathcal{L}(C \cup \{c^*\}, \cdot) - \mathcal{L}(C, \cdot)$
    if $\Delta\mathcal{L} \leq d_{DKL}$ **then**
      break // Stop adding constraints to prevent overfitting
    else
      $C \leftarrow C \cup \{\hat{c}\}$
      $\mathcal{C}_{cand} \leftarrow \mathcal{C}_{cand} \setminus \{\hat{c}\}$
    end
  end
until *convergence of* $\mathcal{L}(C, \{w_k\}, \{\pi_k\})$

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction clearly state that MOCI infers shared constraints and individual preferences from heterogeneous trajectories. Sections 4 and 5 provide empirical evidence (MSE and run-time metrics) that directly support these claims.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The limitations of MOCI algorithm are discussed in Section 6 (Conclusions). Section 4.2 explicitly acknowledges a limitation where limited dataset coverage can lead the algorithm to hallucinate constraints (false positives) in unvisited regions

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The computational complexity is theoretically derived in Section 3.2 and result is presented in figure-6. The detailed analysis/proof for the partition function over feasible paths is provided in Appendix-B.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 4 details the Gridworld environment setup (features, transitions, generation of expert trajectories using Soft Value Iteration), and discusses the critical hyperparameter tuning for the divergence threshold (d_{DKL}). The code along with the run instructions has been provided in the supplementary material for reproducibility of the results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The code along with the instructions has been provided in the supplementary materials. To run the code, please follow the instructions in the README.md file in the supplementary materiel.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4.1 outlines the details of experiment setup and dataset generation (e.g., sampling 10 trajectories from Expert 1 and 10 from Expert 2).

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

Justification: Table-3 in Appendix-C.1 summarizes means $\pm$ standard deviations over 50 independent demo seeds on the heterogeneous GridWorld protocol (d_{DKL}=0.05, 10 EM iterations per run). We report average marginal log-likelihood per trajectory under the fitted model, constraint mean squared error (CMSE), precision and recall for forbidden states, their harmonic mean (F1), and false positive rate (FPR).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: We reported compute resource and software details in section 4.2.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The research focuses on the algorithmic foundations of constraint inference and safe reinforcement learning. It does not involve human subjects, deceptive practices, or inherently malicious applications, and strictly adheres to the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.

- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: We discuss the positive societal impacts of our work in the context of healthcare and personalized medicine (e.g., oncology) in the Future Work section. As foundational algorithmic research, direct negative societal impacts are unlikely; however, we acknowledge that deploying constraint inference algorithms in safety-critical systems carries the risk of agents learning miss-specified constraints if the expert data is deeply flawed.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper proposes a mathematical framework evaluated entirely on synthetic Gridworld environments. It does not release high-risk assets such as pre-trained large language models, image generators, or datasets scraped from the internet.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We properly cite the original papers for the MLCI and ICRL frameworks used as baselines in our experiments. All baseline implementations and evaluations rely on standard, open-source academic libraries.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The code repository for the MOCI framework, including the synthetic Gridworld environment generators and evaluation scripts, will be released open-source upon acceptance. The repository will include comprehensive documentation and instructions for reproducibility.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The experiments are conducted entirely within a simulated Gridworld environment using synthetically generated expert trajectories

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: This paper does not involve crowdsourcing nor research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLM is used only for writing, editing, or formatting purposes. The core method development (MOCI framework, EM approach, and Maximum Entropy IRL) relies on reinforcement learning and probabilistic optimization techniques; it does not involve Large Language Models.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.